

Aspect Based Sentiment Analysis

Roger Baiges Trilla

Guillem Olivart Garrofé

Universitat Politècnica de Catalunya

School of Informatics of Barcelona

Large Language Models

Lluís Màrquez Villodre

May 2026

1 Experimental Setup and Data Diagnostics

The project consists of four required stages: zero-shot prompting, few-shot prompting, synthetic data augmentation, and fine-tuning. We treated all of them as the same constrained generation task: for each bilingual restaurant review, the model must emit a JSON object whose keys belong to the 11 valid aspect categories and whose values belong to the four valid polarities. A prediction is useful only if both the aspect and the polarity are correct, so the task is not just sentiment classification; it is structured extraction with a closed schema.

Our main generator was *Qwen3.5-2B*. We selected it instead of the older model suggested in the starter code because it is a newer small open model, it is multilingual, it has a chat template suitable for instruction following, and it can be run repeatedly on a single 3090-class GPU. This mattered because the project required many full devel-set experiments, not just a single final run. For retrieval-based few-shot prompting we used *Qwen3-Embedding-0.6B*, keeping the generator and retriever in the same model family while using a much smaller model for dense retrieval.

All model selection was performed on `devel.json`, exactly as the assignment defines this split: it is the validation set used to make prompt, decoding, retrieval, and checkpoint decisions. The held-out test set was not used during development. We therefore report all numbers on devel and keep the final conclusions explicitly validation-based.

The official scoring script treats each predicted aspect–polarity pair as one item. We report *m.avg* F1, the global micro-F1 over all predicted and gold pairs, and *M.avg* F1, the average of per-review F1 scores. This distinction matters: *m.avg* rewards recovering frequent labels, while *M.avg* is more sensitive to reviews with few or rare labels. We also track exact match accuracy, because the final output is a complete structured object and one extra or missing label makes the whole review different from the gold JSON. Precision and recall are interpreted at pair level: over-generating plausible aspects hurts precision, while omitting rare aspects hurts recall.

Table 1: Dataset shape. English reviews contain more labels per review, while Spanish dominates the number of examples.

Split	Language	Reviews	Labels	Labels/review	Empty reviews
Train	all	1056	3748	3.55	29
Train	Spanish	713	2360	3.31	29
Train	English	343	1388	4.05	0
Devel	all	132	454	3.44	3
Devel	Spanish	91	285	3.13	3
Devel	English	41	169	4.12	0

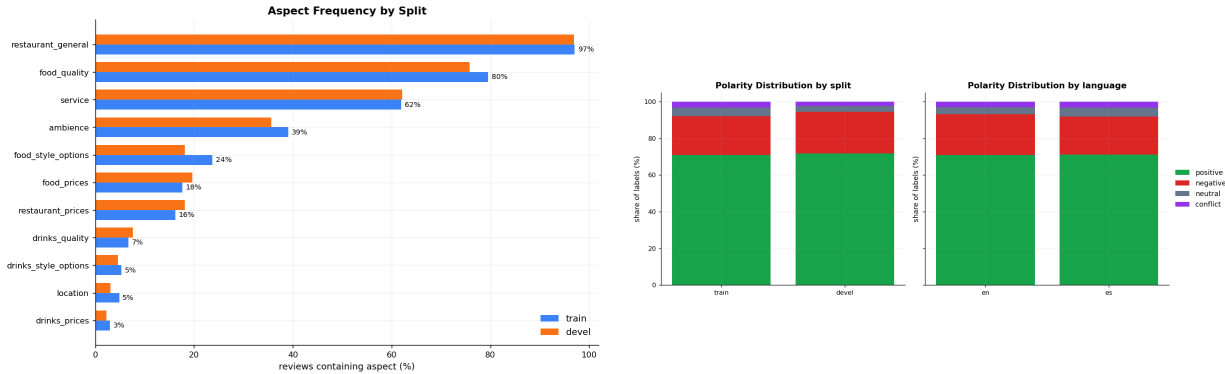


Figure 1: The corpus is dominated by frequent positive labels for `restaurant_general`, `food_quality`, and `service`. Price, drinks, style/options, neutral, and conflict labels form the long tail.

The distribution in Figure 1 drove most design decisions. A model can obtain a non-trivial score by predicting common positive aspects, but the useful part of the task lies in controlling over-prediction and recovering rare boundaries such as restaurant-level prices versus food prices, food quality versus food style/options, and neutral/conflict polarity. This is why we did not optimize only for the highest number of predicted labels: the best system must decide when an aspect is genuinely mentioned, not merely probable in a restaurant review.

The imbalance is not only global but also multilingual. In devel, Spanish contains no `drinks_style_options` gold label, while English contains all six of them. Conversely, most neutral/conflict cases are Spanish. This made language-aware retrieval and language-balanced synthetic generation important: optimizing only the global score can hide failures in the smaller English split or in the Spanish rare-polarity cases.

The number of labels per review also shaped the generation limits. Empty reviews exist, but most examples contain several labels. Therefore, a valid prompt cannot ask for exactly one sentiment, and a valid decoding configuration cannot simply stop after the first pair. At the same time, the gold JSON is short compared with the instruction prompt, which later becomes important for loss masking during fine-tuning. The correct target distribution is compact multi-label JSON, not long natural-language explanations.

Before running LLM experiments we used these observations to define three diagnostic checks for every output file. First, we counted predicted labels against gold labels to detect over-generation or under-generation. Second, we kept raw generations separately from parsed predictions so malformed JSON could be inspected rather than silently discarded. Third, we broke down errors by aspect and language, because a small increase in global F1 can hide large regressions in long-tail labels.

2 Baselines, Zero-Shot Prompting, and Decoding

We first implemented frequency baselines to quantify how much the training distribution alone explains. These baselines do not read the review; they only exploit the marginal label distribution in `train.json`. This is a useful sanity check because the corpus is skewed enough that a naive majority predictor can appear moderately strong.

The strongest legal frequency baseline, `top3_majority`, always predicts the three most frequent aspect–polarity pairs from the training set and reaches 56.24 *m.avg* F1. Predicting more labels increases recall but quickly destroys precision: `top8_majority` sees more gold labels, but its micro-F1 falls to 41.7. The oracle baseline, which uses the gold devel aspects and only guesses polarity by majority class, reaches 71.80 *m.avg* F1. This isolates aspect detection as the harder part: if aspects were known, polarity would be much easier.

Table 2: Frequency baselines. The gap between `top3_majority` and the oracle shows that detecting mentioned aspects is harder than assigning their majority polarity.

Baseline	P	R	<i>M.avg</i> F1	<i>m.avg</i> F1
Empty	0.0	0.0	0.0	0.0
<code>top1_positive</code>	72.0	27.2	36.9	32.4
<code>top3_majority</code>	60.4	56.8	55.6	56.2
<code>top4_majority</code>	51.9	63.4	54.5	55.8
<code>top8_majority</code>	29.8	70.7	40.5	41.7
Oracle aspects + majority polarity	72.6	72.6	72.6	71.8

Table 3: Main zero-shot progression on the development set.

System	Pred.	Gold	P_μ	R_μ	$M.avg$ F1	$m.avg$ F1
Empty baseline	0	454	0.00	0.00	0.00	0.00
top3_majority	396	454	60.35	52.64	55.64	56.24
Zero-shot prompt v1	646	454	39.78	56.61	46.90	46.73
Zero-shot prompt v2	355	454	63.10	49.34	53.86	55.38
Best zero-shot sweep	383	454	74.67	63.00	67.93	68.34
Oracle aspects + majority polarity	454	454	71.81	71.81	72.60	71.81

The first prompt failed mostly by over-generation: it produced 646 labels for 454 gold labels and treated implicit or absent aspects as if they should still be emitted. The second prompt explicitly forbade using **neutral** for “not mentioned”, clarified when **restaurant_general** should appear, and required strict quoted JSON values. This fixed the worst hallucination pattern, but made the model too conservative. We therefore separated prompt design from decoding search and tuned generation parameters after stabilizing the prompt.

The final zero-shot prompt family followed four principles. First, it describes the closed schema in operational terms, especially the difference between absent aspects and neutral polarity. Second, it tells the model to output only JSON, without explanations, markdown, or confidence scores. Third, it treats **restaurant_general** as a real expressed opinion, not as a default label that should always be present. Fourth, it makes empty JSON valid when no aspect is evaluated. These details matter more than wording style: most early failures came from ambiguous task boundaries, not from lack of sentiment knowledge.

The prompt finally used for the rest of the project was **absa_v6**, shown in Appendix A. It was intentionally dataset-aware but not example-biased. Instead of inserting a hand-picked review, it uses aggregate priors from the training split as search hints: **restaurant_general**, **food_quality**, **service**, and **ambiance** are named as common categories, while drinks, prices, style/options, and location are described as valid but evidence-dependent categories. This was the key compromise between v1 and v2. The model is encouraged to recover the common labels that devel reviews often contain, but every label still needs textual support.

Table 4: Main design elements of **absa_v6**. The prompt is shown in full in Appendix A.

Prompt block	Purpose
Strict output contract	Forces exactly one JSON object with quoted valid aspect and polarity strings.
Dataset-aware priors	Raises recall for frequent aspects without adding demonstrations or leaking devel examples.
Recall rule	Prevents the overly conservative v2 behavior by reminding the model that long reviews often contain several labels.
Neutral/conflict rules	Separates explicit neutral sentiment from absent aspects, and reserves conflict for mixed sentiment on the same aspect.
Aspect mapping hints	Turns ambiguous wording such as value, portions, waiting, decor, wine list, or parking into the correct closed aspect.
Completeness checklist	Makes the model do one final pass over food, service, ambiance, prices, drinks, location, and unsupported labels.

Later prompt variants, especially **absa_v8**, tried to make the wording more balanced, but they did not replace **absa_v6** as the project anchor. The reason is practical: **absa_v6** was the prompt used for the strongest hyperparameter search, the few-shot experiments, and the fine-tuning format. Keeping the same prompt across stages made comparisons meaningful. Otherwise an apparent improvement from few-shot or fine-tuning could partly be a prompt-format change rather than a true method improvement.

The best zero-shot region was not the official Qwen sampling recipe. For this extraction task, the best setting was more conservative: temperature 0.65, top-p 0.75, top-k 20, presence penalty

2.0, repetition penalty 1.0, and max-new-tokens 512. This improved the official-like setup from 55.87 to 68.34 *m.avg* F1.

Table 5: Representative decoding sweep after the prompt was fixed to `absa_v6`.

Config	Temp.	Top-p	Top-k	PP	<i>M.avg</i> F1	<i>m.avg</i> F1
Official-like	1.00	1.00	20	2.0	55.53	55.87
No presence penalty	1.00	1.00	20	0.0	56.74	59.47
Conservative	0.70	0.80	20	1.5	65.20	66.12
Refined	0.70	0.80	20	1.8	66.64	68.18
Best single run	0.65	0.75	20	2.0	67.93	68.34

The sweep shows that the model needs restraint, not open-ended diversity. Presence penalty is useful before fine-tuning because it discourages repetitive lists and helps the model stop adding the same common labels, but too much sampling makes the model enumerate plausible restaurant aspects instead of extracting only the expressed ones. We also checked narrower configurations close to greedy decoding, but before fine-tuning they tended to lose recall: the base model had not yet internalized the extraction format, so a small amount of stochasticity helped recover labels that greedy decoding missed.

Optuna was useful mostly as an organized local search rather than as a fully automated optimizer. Because each devel run was expensive, we did not run a huge combinatorial grid over prompts, *K*, temperature, top-p, and top-k. Instead, we fixed a good prompt, tested coherent decoding regions, inspected their errors, and then refined around the best region. This kept the number of experiments linear and interpretable: every new run answered a specific question about precision, recall, or stability.

We also tested Qwen thinking mode, but it was unusable for the main pipeline: in an 8-example pilot with a 4096-token budget, only 2 generations closed `<think>`, 6 hit the limit, and the run produced only 2 predicted labels over 22 gold labels. The model was not mainly looping; it was spending the whole budget deliberating about definitions and never emitting the final JSON.

Malformed JSON was not the dominant zero-shot failure after prompt v2, but we kept a conservative parser repair step for common JSON-like outputs such as unquoted polarity values. The raw string is stored, then the evaluator parses a normalized prediction. The repair only accepts valid aspect names and valid polarity values; it does not invent labels and it does not map arbitrary free text to the closest aspect. This improved robustness without changing the semantic task. After this point, the dominant errors were semantic: missing rare aspects, confusing price scopes, and collapsing neutral/conflict into positive or negative.

3 Few-Shot Prompting

The few-shot stage used `train.json` demonstrations only; no devel examples were ever used as prompt examples. We compared random fixed examples, pure dense retrieval, a manually curated hard bank, a hard/retrieved mix, and an ABSA-aware MMR reranker. The final strategy, `absa_mmr`, retrieves dense candidates with Qwen3-Embedding-0.6B and reranks them using semantic similarity, same-language preference, aspect-cue coverage, rare-label usefulness, length/label-count fit, and a diversity term. This was designed to avoid the common failure of semantic retrieval: selecting several near-duplicate positive restaurant reviews that do not teach rare aspects.

The selection problem is more important than it may look. A random few-shot prompt often teaches the model the majority distribution again, while a purely manual bank cannot adapt to the query. Dense retrieval is much better, but nearest neighbors can be too similar to each other: for example, a query about a positive dinner can retrieve several examples about positive food and service while missing the price or style boundary that actually matters. Our reranker therefore scores a candidate not only by similarity to the current review, but also by how much new ABSA

information it adds relative to the examples already selected.

Table 6: Exploration-stage few-shot comparison for $K = 5$, seed 42. Retrieval quality matters more than simply adding examples.

Method	Pred.	P_μ	R_μ	$M.avg$ F1	$m.avg$ F1
dense_topk	381	80.4	68.4	72.4	72.3
absa_mmr	388	79.1	68.5	72.0	71.7
hard_mix	392	74.7	65.7	68.4	68.3
random_fixed	412	74.7	65.8	67.5	67.4
manual_fixed_hard	412	73.2	65.8	67.5	67.4

The $K = 5$ exploration suggested that dense retrieval is already strong, but the final choice should not be based on one seed and one K . We therefore ran a confirmation stage over stronger K values and multiple seeds. This is why the final few-shot result is not simply the best single exploration run: it is the setting that remained strong and stable when randomness changed.

Table 7: Few-shot confirmation stage. Values are means over seeds 101, 202, and 303.

Method	K	P_μ	R_μ	$M.avg$ F1	$m.avg$ F1	Std $m.avg$
absa_mmr	8	79.60	69.02	73.85	73.93	0.50
absa_mmr	12	78.00	70.04	73.92	73.81	0.79
dense_topk	12	77.98	69.68	74.19	73.59	0.98
dense_topk	10	78.29	68.58	73.31	73.11	0.51

absa_mmr with $K = 8$ was selected as the best few-shot system because it had the best mean $m.avg$ F1 and the lowest variance among the top configurations. Increasing to $K = 12$ slightly improved recall but made the prompt less stable. The confirmed few-shot runs were mechanically clean: 100% JSON parse success, no token-limit failures, and about 20 output tokens on average. The remaining errors were therefore semantic rather than formatting issues, mostly concentrated in **restaurant_prices**, **food_prices**, **food_style_options**, drinks aspects, and neutral/conflict polarity.

The language split exposed an additional retrieval effect. The selected $K = 8$ ABSA-MMR configuration averaged 73.77 $m.avg$ F1 in Spanish and 74.22 in English across the three seeds. English precision was higher, about 85.0 micro precision, but recall was lower because English reviews contain more labels per review. This justified keeping same-language matching as a soft score rather than a hard filter: cross-language examples can still be useful, but the default demonstrations should match the review language when possible.

The manual hard bank was still valuable as an analysis tool. It contained examples for empty reviews, mixed polarity, explicit neutral opinions, price/value complaints, style/options cues such as portions and variety, and cases where a review praises the restaurant globally but criticizes one concrete aspect. However, using only these fixed examples made the prompt less query-specific. The best compromise was to let retrieval choose the final demonstrations and encode the manual insights as reranking preferences rather than as a static prompt.

This stage also established an important ceiling for inference-only methods. Few-shot prompting moved from 68.34 to 73.93 $m.avg$ F1, a clear gain, but the model still missed many long-tail labels. The remaining gap was no longer a formatting or demonstration-selection problem. It required changing the model weights so that the ABSA boundaries became part of the model’s own decision function.

4 Fine-Tuning Design

Prompting and retrieval improved the model, but still left a large gap to the oracle baseline. We therefore fine-tuned Qwen3.5-2B with parameter-efficient adapters. The most important training detail was loss masking: the system prompt, user prompt, and any instruction text are masked with label value -100 , so the loss is computed only on the assistant JSON answer. Without this, the prompt is much longer than the gold JSON and the model would spend most of its capacity learning to reproduce instructions instead of learning ABSA decisions.

We trained without few-shot examples in the SFT prompt. Few-shot prompting is an inference-time retrieval strategy; including demonstrations during training would change the input distribution and make the final model depend on long prompts. The final SFT format therefore matches the desired zero-shot inference format: instruction plus review text in, compact JSON out.

Both LoRA and QLoRA used rank 16, alpha 32, dropout 0.05, all linear Qwen projection modules, learning rate 10^{-4} , cosine schedule, 5% warmup, weight decay 0.01, max length 3072, and generative validation every 50 optimizer steps. The micro-batch was limited by GPU memory to 2 examples per device, so the effective batch size was controlled with gradient accumulation. QLoRA loads the base model in 4-bit and was the best training variant, reaching 87.35 *m.avg* F1. The max length was chosen from a token-budget notebook: `absa_v6` plus compact gold JSON fits all train/devel examples below 3072 tokens, while dynamic padding avoids paying this length for every batch.

We did not create an additional validation split from `train.json`. The assignment explicitly defines `devel.json` as the split for development decisions, and the training set is already small. Holding out more train examples would reduce the supervision available for rare labels. Instead, the code trains on all of `train.json`, evaluates on devel every 50 optimizer steps, and keeps the checkpoint with the best generated ABSA F1. The final test set, once released, should be used only once with the selected checkpoint.

Table 8: Training choices that differ from a minimal fine-tuning script.

Design choice	Reason
Prompt loss masking	Prevents the long instruction prompt from dominating the short JSON target loss.
No few-shot examples in SFT	Keeps training aligned with zero-shot fine-tuned inference and avoids dependence on retrieved demonstrations.
Generative validation	Selects checkpoints by actual JSON extraction quality instead of teacher-forced token loss.
Greedy fine-tuned inference	Removes sampling variance after the model has learned the structured output distribution.
All-linear LoRA targets	Adapts attention and MLP projections without full-model training cost.

Loss masking is the main difference between a task-specific SFT setup and a naive chat fine-tuning setup. The prompt contains the complete aspect definitions, output constraints, and the review text; the target completion is usually a short JSON object. If the full sequence were supervised, most tokens in the loss would belong to the repeated prompt template. The model could obtain a low loss by learning to continue or copy instructions while paying relatively little attention to the scarce answer tokens. Masking the prompt makes every gradient update focus on the actual extraction decision: which aspect keys to include and which polarity value to assign.

This also explains why evaluation loss is not enough for model selection. A checkpoint can have slightly lower teacher-forced token loss because it assigns high probability to common JSON syntax, yet still produce worse generated outputs after decoding. ABSA is evaluated after parsing the generated object, so a single extra key, missing key, or wrong polarity has a discrete effect that token loss does not fully capture.

Table 9: Best fine-tuned checkpoints on `devel.json`.

Run	Step	Pred.	OK	P_μ	R_μ	$M.avg$ F1	$m.avg$ F1
LoRA on train	450	437	385	88.10	84.80	84.44	86.42
QLoRA on train	400	439	390	88.84	85.90	85.29	87.35
Augmented PEFT on train+synthetic	450	449	394	87.75	86.78	85.17	87.26

Checkpoint selection by validation loss would have chosen weaker models. For QLoRA, the minimum validation loss occurred at step 350 with 85.71 $m.avg$ F1, while the best generative checkpoint was step 400 with 87.35. The difference is even clearer in the augmented run: the minimum loss was step 250 with 84.13 $m.avg$ F1, but the best generated ABSA output was step 450 with 87.26. The loss values at steps 250 and 450 were almost identical, yet the task F1 differed by more than 3 points. We therefore selected checkpoints by full greedy generation on the development set, using $m.avg$ F1 as the primary metric and $M.avg$ F1 as the tie-breaker.

Table 10: Validation loss does not select the best ABSA checkpoint.

Run / checkpoint	Step	Eval loss	$M.avg$ F1	$m.avg$ F1
QLoRA, minimum loss	350	0.1147	83.74	85.71
QLoRA, best generated F1	400	0.1204	85.29	87.35
Augmented, minimum loss	250	0.1175	82.66	84.13
Augmented, best generated F1	450	0.1184	85.17	87.26

After fine-tuning, greedy decoding became the best inference policy. The sampling settings that helped zero-shot prompting were useful because the base model was under-calibrated for this extraction format and needed some exploration to recover missing labels. After SFT, the model has already learned the target JSON distribution; stochastic sampling mainly injects variance, extra labels, and polarity instability. Since ABSA extraction is a deterministic structured-output task, greedy decoding gives better reproducibility and preserves the decision boundary learned during fine-tuning.

For this reason, we did not reuse the zero-shot best decoding parameters for the final fine-tuned model. The zero-shot setting with temperature 0.65 and top-p 0.75 was a compensation mechanism for a base model that had not been trained on the ABSA schema. Once the model is supervised on gold JSON, the desired behavior is the most likely structured answer, not a sample from a distribution of plausible answers. Keeping sampling after SFT would make the same review produce slightly different label sets across runs, which is undesirable for a leaderboard-style extraction task.

5 Targeted Synthetic Data Augmentation

The assignment suggested generating many random synthetic reviews, but our best QLoRA checkpoint was already well calibrated: 439 predicted labels for 454 gold labels, no empty predictions on non-empty reviews, and no hallucinated predictions on empty-gold reviews. A large naive synthetic dataset risked damaging this calibration. We therefore generated a smaller targeted augmentation set of 220 examples.

The synthetic generation plan was driven by real QLoRA errors. We first ran the best QLoRA model on the training set and used its mistakes to identify hard regions: restaurant-level price judgments, food price versus restaurant price, food style/options, rare drinks aspects, ambience boundaries, and neutral/conflict polarity. Each train example received a difficulty score based on false negatives, false positives, polarity mistakes, rarity of the gold labels, and language. The development set was used only for aggregate diagnosis and final comparison; no devel review text was sent to the API or paraphrased.

Each synthetic example was generated by a strong API model from a controlled plan: target language, difficulty bucket, exact target gold dictionary, aspects to avoid, official aspect and polarity definitions, and three style examples selected from `train.json`. These style examples were not random; they were chosen to match language, share target aspects or aspect-polarity pairs, include cases difficult for the current model, and avoid excessive reuse. The API was not asked to label an arbitrary review; it was asked to write a natural review that realizes a specific gold JSON. This direction is important because it gives us direct control over the intended supervision signal.

The API response also had to include support spans, and a local validator required exact target-gold agreement, valid labels only, support-span presence, no annotation-like phrasing, and low n-gram similarity to train/dev examples. We accepted quality over quantity: rejected generations were not kept simply to reach a round number. This is why the final file contains 220 accepted examples rather than a large unfiltered synthetic corpus.

Table 11: Final synthetic dataset composition.

Bucket	Count	Language / shape	Count
Price/value boundary	56	Spanish	148
Food style/options boundary	36	English	72
Ambience/location/service boundary	30	0-label hard negatives	11
Rare drinks aspects	24	2-label examples	36
Neutral/conflict polarity	40	3-label examples	143
Hard negatives / empty-gold	14	4-label examples	27
Natural regularizers	20	Total accepted	220

The final training file was `train_plus_synthetic_v1.shuffled.json`. Shuffling was intentional: the unshuffled file appended all 220 synthetic examples after the 1056 original examples, which could make late training steps disproportionately synthetic. The shuffled version preserves the exact same 1276-example multiset while distributing synthetic examples throughout the training order.

This design deliberately avoids using synthetic data as a replacement for the original corpus. The original train examples remain the majority and preserve the natural dataset distribution, while synthetic examples act as targeted local corrections. This is especially important for ABSA because nearby labels are easy to confuse: adding too many price examples can move the model from missing `restaurant_prices` to overusing price labels in food-specific contexts.

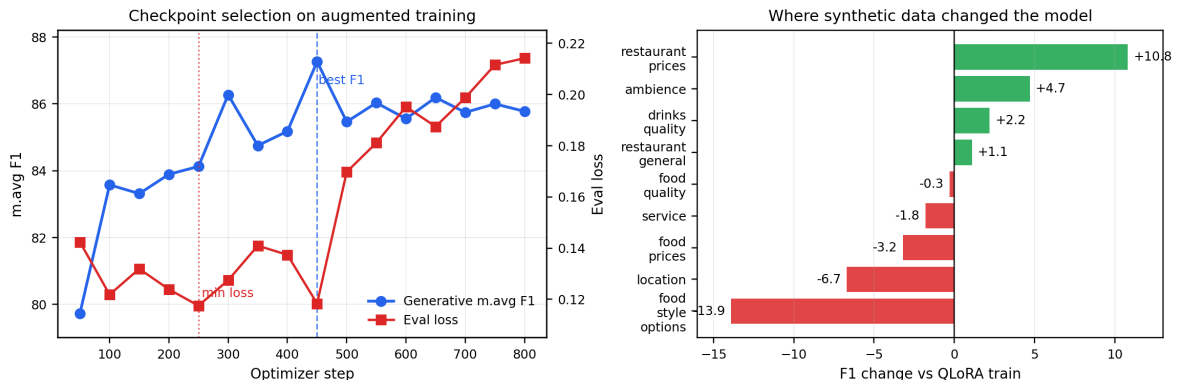


Figure 2: Synthetic augmentation did not improve the global best F1, but it changed useful boundaries. Left: validation loss would select a weaker checkpoint than generative F1. Right: the augmented model improves `restaurant_prices` and `ambience`, but hurts nearby style/price boundaries.

The augmented run is best interpreted as a controlled negative result. It did not beat

the original QLoRA checkpoint globally: 87.26 versus 87.35 *m.avg* F1. However, it improved exact match accuracy from 73/132 to 76/132 and recovered more gold labels overall (394 versus 390 correct pairs). More importantly, the local effect matched part of the intended target: **restaurant_prices** improved from 63.2 to 73.9 aspect-F1, **ambience** from 81.3 to 86.0, and **conflict** polarity from 28.6 to 42.1. These gains were offset by degradation in nearby categories, especially **food_style_options** (79.2 to 65.2), **food_prices** (74.1 to 70.8), and **neutral** polarity (44.4 to 40.0). This shows that synthetic ABSA data can move decision boundaries, but controlling one boundary can destabilize another.

The most informative part is not the tiny global difference; it is the direction of the local changes. The augmentation succeeded where it was explicitly targeted, especially restaurant-level price/value and ambience cues. The cost appeared in adjacent labels that share vocabulary with the targets. For example, complaints about price can refer to the restaurant globally, to a specific dish, or to value-for-money in general. Similarly, food style/options overlaps with food quality when a review mentions portion size, variety, or presentation together with taste. The synthetic set therefore improved some intended boundaries but introduced pressure on related ones. This is exactly the type of trade-off that a final report should surface rather than hide.

Table 12: Main local changes after synthetic augmentation, compared with the QLoRA train-only checkpoint.

Category	QLoRA F1	Aug. F1	Change	Interpretation
restaurant_prices	63.2	73.9	+10.8	target boundary improved
ambience	81.3	86.0	+4.7	explicit atmosphere recovered
conflict polarity	28.6	42.1	+13.5	rare polarity improved
food_prices	74.1	70.8	-3.2	price boundary shifted
neutral polarity	44.4	40.0	-4.4	still under-controlled
food_style_options	79.2	65.2	-13.9	style/quality boundary degraded

6 Language Robustness and Final Comparison

The final models were balanced across languages despite the split imbalance. Spanish has more reviews, while English reviews contain more labels per example, making exact match harder. The best QLoRA checkpoint is slightly stronger in Spanish by *m.avg* F1, while the augmented model improves exact match in both languages.

Table 13: Language-level results for the two strongest fine-tuned systems.

Model	Language	Gold	Pred.	<i>M.avg</i> F1	<i>m.avg</i> F1	Exact
QLoRA train	Spanish	285	282	84.87	87.83	56/91
QLoRA train	English	169	157	86.22	86.50	17/41
Augmented PEFT	Spanish	285	286	84.42	87.92	58/91
Augmented PEFT	English	169	163	86.85	86.14	18/41

The language-level numbers show that the final system is not simply overfitting to the majority Spanish split. The QLoRA model reaches 87.83 *m.avg* F1 in Spanish and 86.50 in English. The augmented run slightly improves Spanish *m.avg* and exact match, but slightly reduces English *m.avg* while still improving English exact match. This is consistent with the synthetic design: it was targeted at hard boundaries, not at a uniform language-level gain. Because English reviews contain more labels per review, one missing label has a stronger effect on exact match than on pair-level F1.

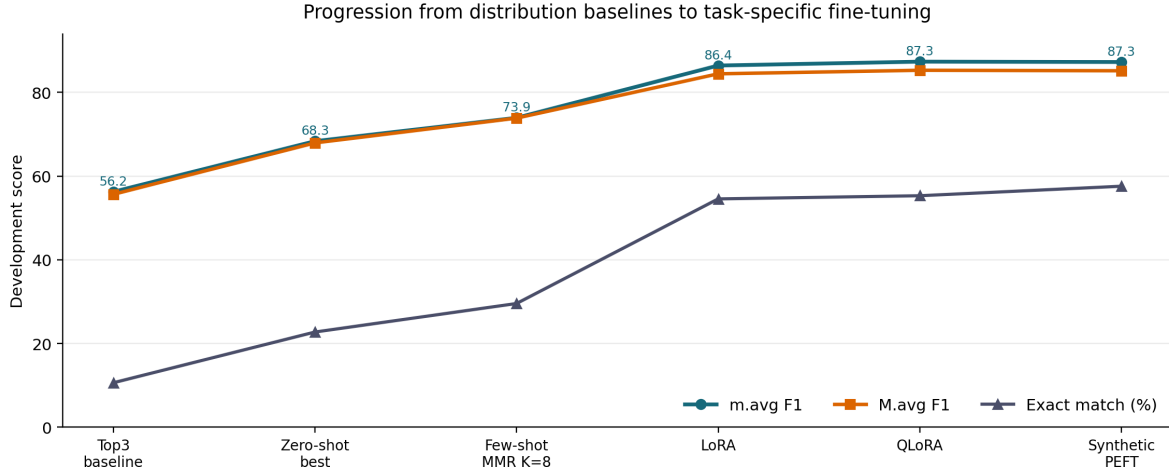


Figure 3: Project-level progression. Prompting and decoding provide the first large gains, few-shot retrieval improves semantic coverage, and fine-tuning produces the decisive jump.

Table 14: Main development-set results across the project.

System	P_μ	R_μ	$M.avg$ F1	$m.avg$ F1	Exact
<code>top3_majority</code> baseline	60.35	52.64	55.64	56.24	14/132
Best zero-shot sweep	74.67	63.00	67.93	68.34	30/132
Best few-shot, <code>absa_mmr</code> , $K = 8$	79.60	69.02	73.85	73.93	39/132
LoRA on train	88.10	84.80	84.44	86.42	72/132
QLoRA on train	88.84	85.90	85.29	87.35	73/132
Augmented PEFT on train+synthetic	87.75	86.78	85.17	87.26	76/132

The main lesson is that each stage solved a different bottleneck. Prompt engineering and decoding fixed formatting and over-generation; ABSA-aware few-shot retrieval improved coverage without fine-tuning; supervised adapters produced the large jump by internalizing the extraction format; generative-F1 checkpointing was necessary because token loss did not align with the evaluation metric; and synthetic data was useful diagnostically even though it did not improve the global score. The final system is therefore the QLoRA checkpoint selected by greedy generative validation, with the augmented run as evidence that targeted synthetic data can improve specific rare boundaries but must be controlled carefully to avoid shifting errors into related categories.

The remaining errors are no longer parser failures or broad hallucination. They are annotation-boundary errors: whether “expensive for what it offers” is a restaurant price or a food price, whether generous portions are food quality or style/options, whether a mixed review should be conflict or negative overall, and whether a short positive sentence implies `restaurant_general` only or also a concrete aspect. These are exactly the decisions where a small local model benefits most from task-specific fine-tuning, but also where synthetic data must be generated with very precise controls.

If we had one more iteration, the most promising direction would not be a larger random synthetic set. The results suggest a narrower plan: generate fewer but more contrastive minimal pairs for the boundaries that moved in the wrong direction, especially `food_prices` versus `restaurant_prices`, `food_quality` versus `food_style_options`, and neutral versus conflict. Each pair should keep the surface review similar while changing only the intended aspect or polarity. That would train the model on the exact distinctions that still fail, instead of increasing the amount of generic restaurant text.

A Prompt absa_v6

The following is the exact prompt file used as the project anchor for zero-shot tuning, few-shot prompting, and fine-tuning format. It is included here because small wording changes had measurable effects on recall and over-generation.

```
NAME: absa_v6

DESCRIPTION:
Dataset-aware zero-shot ABSA prompt tuned for higher recall using aspect-frequency priors, while keeping
↳ strict JSON and avoiding unsupported labels.

SYSTEM PROMPT:
You are a strict multilingual Aspect-Based Sentiment Analysis extractor for restaurant reviews. Your goal
↳ is to recover every explicitly supported aspect-polarity pair in the text. Use the dataset priors as
↳ guidance for what is likely to be mentioned, but never guess unsupported labels. Return only one
↳ compact JSON object.

USER PROMPT TEMPLATE:
Analyze the restaurant review and return a JSON object mapping only the explicitly evaluated aspects to
↳ their sentiment polarities.

Output format:
- Return exactly one valid JSON object.
- Use double-quoted aspect names and double-quoted polarity strings.
- Example: {"restaurant_general": "positive", "service": "negative"}.
- Do not output markdown, explanations, comments, lists, or extra text.

Dataset-aware extraction strategy:
- The dataset is imbalanced. Some aspects appear much more often than others. Use that information to
↳ search carefully, but do not force labels that are not supported.
- `restaurant_general` appears in almost every review, about 97%. If the review gives an overall opinion,
↳ recommendation, satisfaction, disappointment, or return intention, include it.
- `food_quality` is very common, about 80%. If the review talks about the food at all with sentiment,
↳ include it.
- `service` is common, about 62%. If the review says anything positive or negative about staff, attention,
↳ speed, kindness, waiting, reservations, or treatment, include it.
- `ambience` appears in a meaningful share of reviews, about 39%. Include it when the review talks about
↳ atmosphere, noise, music, decor, cleanliness, space, crowd, or the type of people there.
- `food_style_options` appears when the review talks about menu variety, menu length, few/many options,
↳ presentation, originality, portion size, or dietary options.
- `food_prices` appears when the review complains about food price or food value, including the menu price.
- `restaurant_prices` appears when the review complains about the restaurant being expensive, cheap,
↳ overpriced, good value, or bad value in general.
- `drinks_quality` appears when the review talks about drink quality, including stale, flat, spoiled, bad,
↳ or low-quality drinks or wine.
- `drinks_style_options` appears when the review talks about drink variety, wine list, beverage options,
↳ or lack of drink choices.
- `drinks_prices` appears when the review explicitly talks about drink prices, such as water being too
↳ expensive, cheap wine, or expensive wine.
- `location` appears when the review talks about location, access, surroundings, neighborhood, parking,
↳ distance, or view.

Important recall rule:
- If the review is not extremely short or empty, try to recover at least 3 labels when the text clearly
↳ supports them.
- Do not invent labels to reach 3.
- If the review is very short or lacks information, fewer than 3 labels is fine.
- If the review is long and touches many topics, 5 or 6 labels may be appropriate.
- The objective is to include all supported labels, not to keep the output small.

Core extraction rules:
- Scan sentence by sentence and clause by clause.
- Include every aspect that the review explicitly evaluates.
- Do not stop after the first obvious label.
- Do not fill absent categories with neutral.
- Neutral means the aspect is explicitly evaluated as average, normal, okay, fine, acceptable, or neither
↳ good nor bad. It never means absent or implied.
- Conflict only when the same aspect has both clear positive and clear negative sentiment. Otherwise
↳ choose the dominant polarity.
- Strong lexical cues count as evidence even if the exact aspect word is not repeated.
- A mention without evaluation is not enough.
- Do not infer rare aspects unless the text explicitly supports them.
```

```

- The review may be in English or Spanish, but output aspect names and polarities only in English.

Valid aspects:
- restaurant_general
- restaurant_prices
- food_quality
- food_prices
- food_style_options
- drinks_quality
- drinks_prices
- drinks_style_options
- ambience
- service
- location

Valid polarities:
- positive
- negative
- neutral
- conflict

Polarity guidance with examples:
- positive: "I really enjoyed it", "I loved the food", "me encantó", "muy bueno", "excellent".
- negative: "I don't like it", "could be better", "very bad", "no me gustó", "pésimo".
- neutral: "normal", "okay", "fine", "acceptable", "regular", "ni fu ni fa".
- conflict: the same aspect is praised and criticized in the same review.

Aspect mapping hints:
- If the review says the restaurant overall was good, bad, recommended, disappointing, or worth returning
  ↪ to, use `restaurant_general`.
- If the review talks about food quality, taste, cooking, freshness, or portions, use `food_quality`.
- If it talks about staff, service, waiting, or attention, use `service`.
- If it talks about atmosphere, noise, music, decor, crowd, or comfort, use `ambience`.
- If it talks about the menu being short, limited, poor, varied, or full of options, use
  ↪ `food_style_options`.
- If it complains about the food or menu being expensive or poor value, use `food_prices`.
- If it complains about the restaurant being expensive or good value in general, use `restaurant_prices`.
- If it complains about the quality of drinks or wine being stale, flat, spoiled, or bad, use
  ↪ `drinks_quality`.
- If it talks about drink options, wine list, or beverage variety, use `drinks_style_options`.
- If it talks about drink prices such as expensive water or cheap/expensive wine, use `drinks_prices`.
- If it mentions access, location, surroundings, or parking, use `location`.

Decision policy:
- Use only the exact aspect names and polarity values listed above.
- Do not invent labels such as dish_quality, staff, menu, experience, or atmosphere if they are not one of
  ↪ the allowed aspects.
- Do not default to positive just because positive is common in the dataset. Only use positive when the
  ↪ text is clearly positive.
- Do not use neutral as a filler for missing aspects.
- If several statements discuss the same aspect, return one final polarity for that aspect.
- The best answer is complete and supported.
- If no valid aspect is explicitly evaluated, return {}.

Before finalizing, do a quick completeness check:
1. Did I catch the overall opinion if present?
2. Did I catch food, service, ambience, prices, drinks, and location if explicitly mentioned?
3. Did I miss any aspect that has clear sentiment?
4. Did I avoid adding unsupported labels just to reach a minimum count?
If the answer to 3 is yes, include the label. If the answer to 4 is yes, remove the unsupported label.

Review language: {language}
Review text:
{text}

```